Máster en Mecatrónica
3er cuatrimestre
“Desarrollo de un Sistema Mecatrónico Industrial)

Guía para el desarrollo

Curso 2026-2027

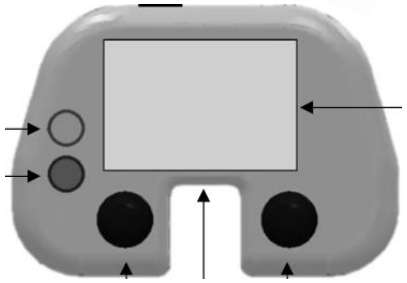


PC (Win o Linux)

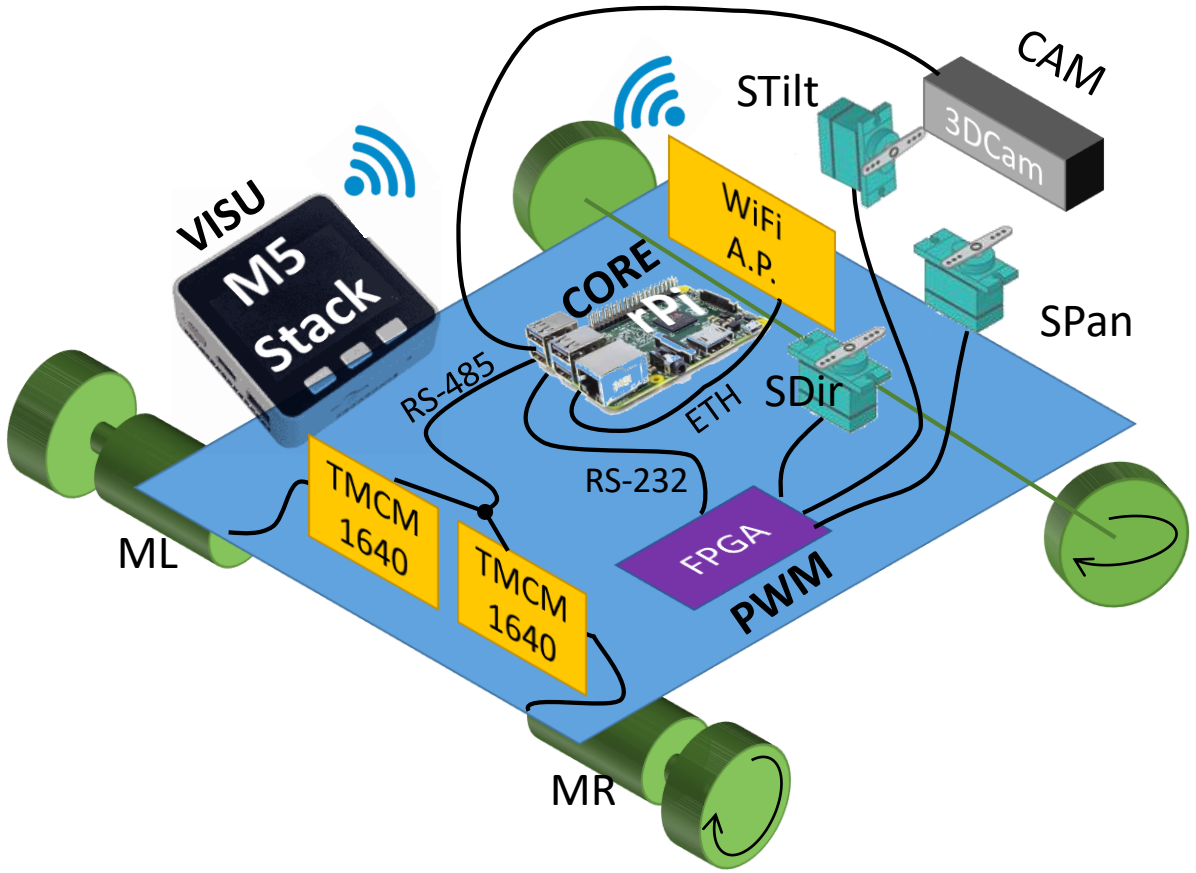
USER-PHONE



Móvil Android



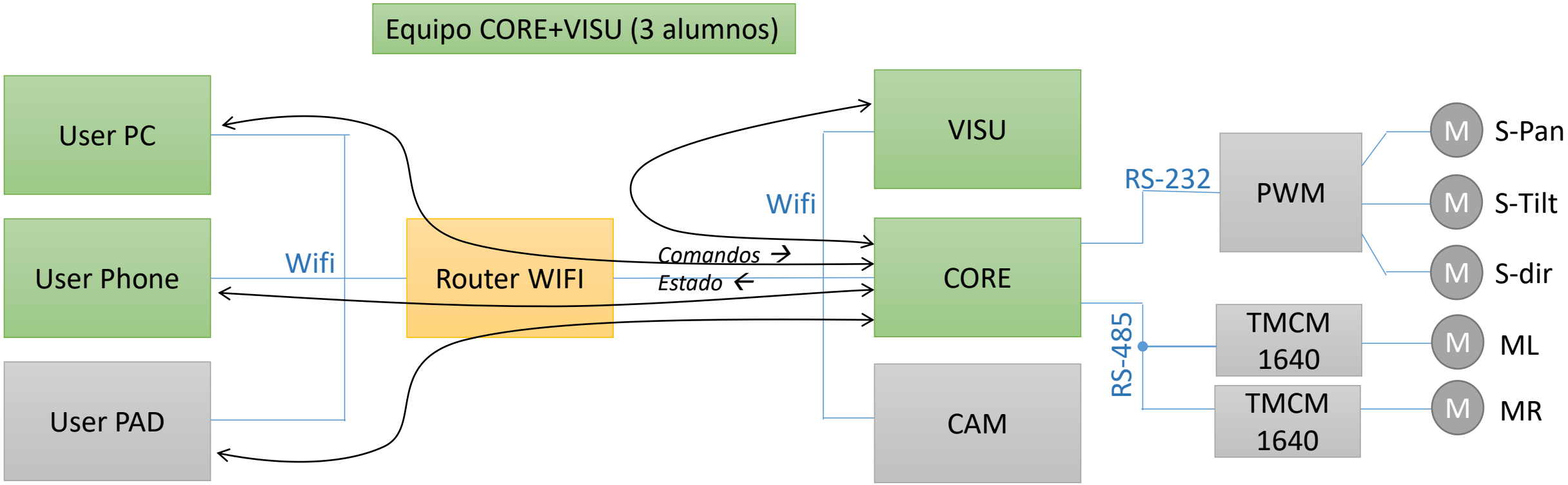
USER-PAD



TAREA 1: Dar nombre a todos los elementos
Usar el mismo nombre en toda la programación y la documentación

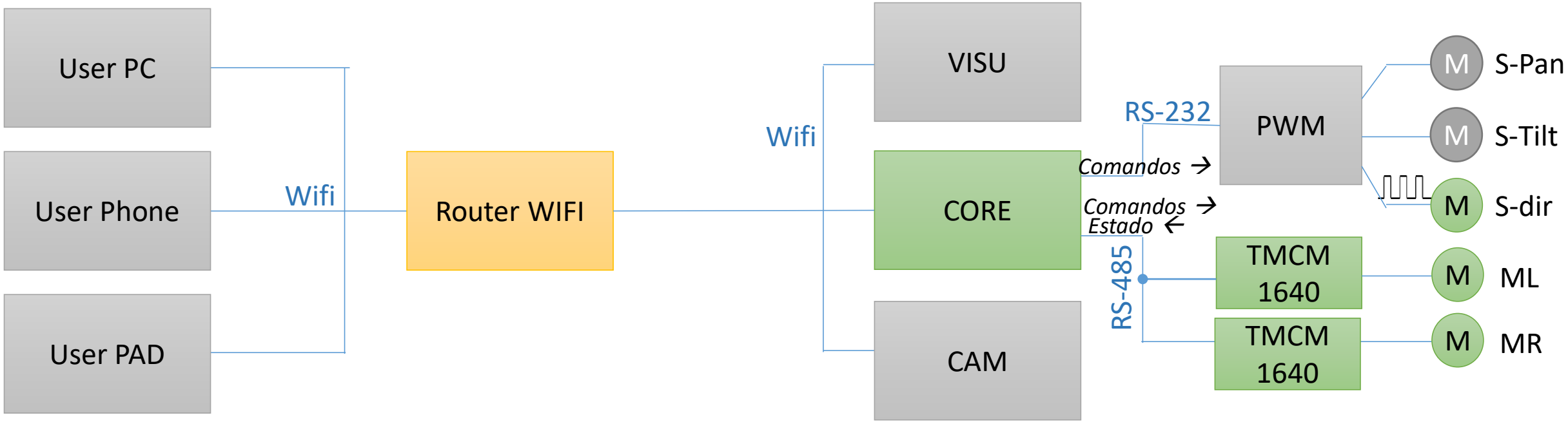
Equipo	Plataforma	S.O.	Código a ejecutar	Lenguaje	Configuración
CORE	Raspberry Pi 5	Linux Raspbian	Gestión general, comunicaciones con subsistemas	C++	Qt modo consola
PATH	Raspberry Pi 5	Linux Raspbian	Gestión de trayectorias	C++	Qt modo consola
USER-PAD	ESP32-S3-DEVKITC-1	Ninguno	Adquisición de sensores joystick, visualización imagen	C++	vsCode + PlatformIO + pioArduino
PWM (servos rotación)	FPGA	Ninguno	Generación señales PWM para posicionamiento de servos	VHDL	
VISU	M5-Stack	Ninguno	Visualización estado carro, interfaz muy básico	C++ Arduino	Arduino IDE ó vsCode
USER-PC	PC	Win o Linux	Interfaz usuario para visualización completa de estado, comandos y configuraciones	C++	Qt modo GUI o Cliente Web
USER-PHONE	Tfno móvil	Android	Interfaz usuario para visualización sencilla de estado, comandos	C++	Qt modo GUI o Cliente Web

Interfaz auxiliar: no debe ser necesario para el funcionamiento del sistema



Diseño mecánico: Chasis carro y soporte motores
Diseño electrónico: Alimentación por baterías
Programación y control:
- Gestión CORE (servidor TCP, mensajes, archivo configuración, etc.)
- Visualización básica en carro (VISU) y en User PC/Phone

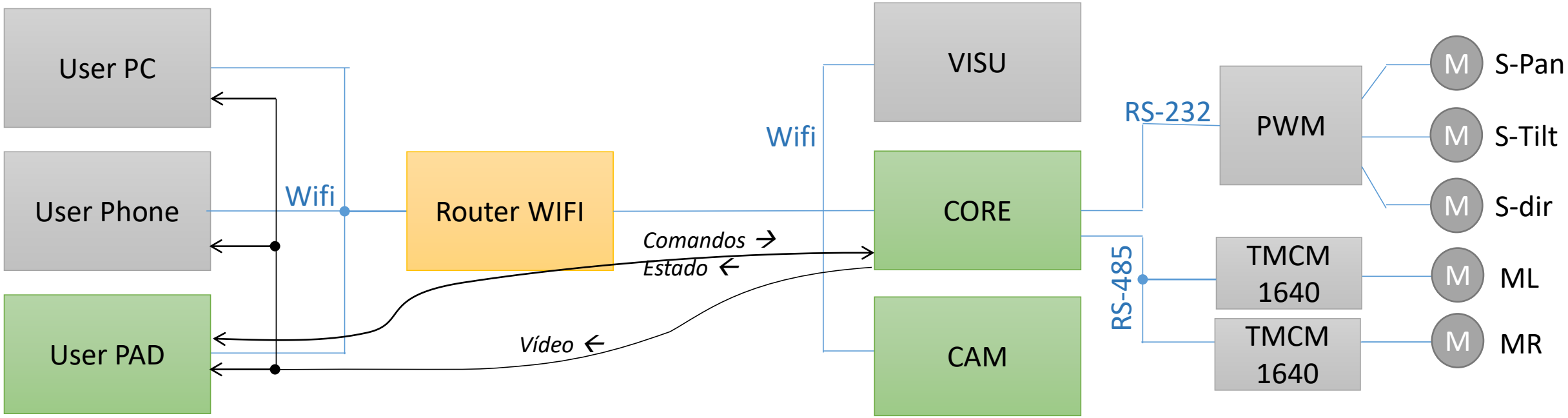
Equipo PATH (2 alumnos)



Diseño mecánico: Dirección carro
Diseño electrónico: Servo dirección desde PWM (comm. Serie con FPGA)
Programación y control:
- Cálculo, generación e info trayectorias
- Movimientos coordinados ML, MR, Sdir

Equipo USER-PAD

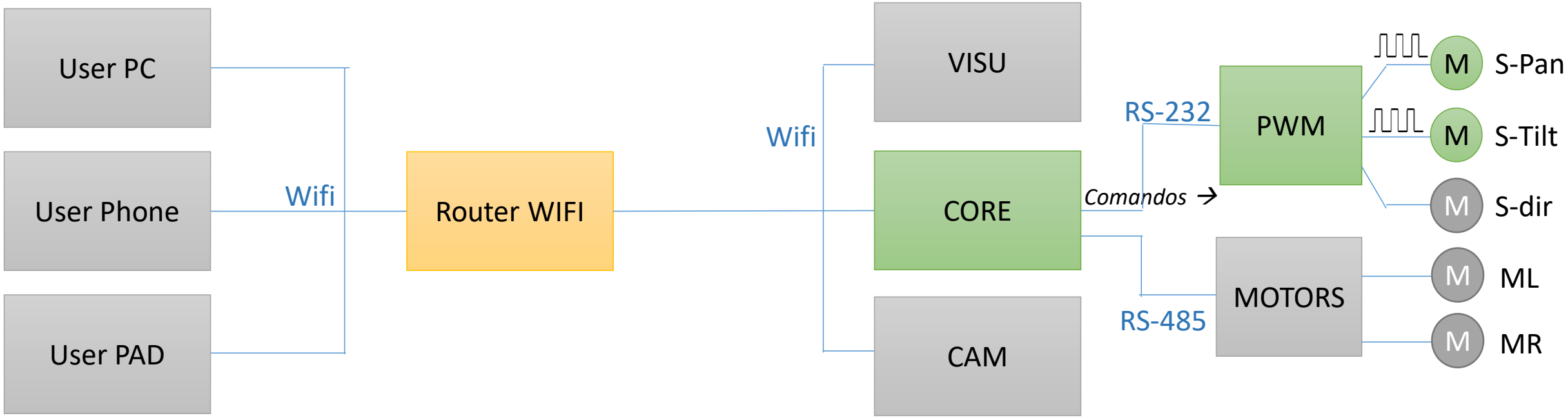
(REALIZADO PROFESOR)



Diseño mecánico: Carcasa User PAD y soporte CAM
Diseño electrónico: Tarjeta User PAD
Programación y control:
- Generación de movimientos a partir de joystick-botonera
- Envío de imágenes 3D para visualización

(REALIZADO PROFESOR)

Equipo PWM-PanTilt (3 alumnos)



Diseño mecánico: Sistema pan-tilt y montaje del sensor 3D
Diseño electrónico: Generación de señales PWM (FPGA) para servomotores
Programación y control:
- Comunicaciones serie CORE-PWM
- Log manager para todos los sistemas

Programación orientada a objetos y eventos

- CORE: programa modo consola

main.cpp

```
#includes necesarios

int main(int argc,char* argv[])
{
   QCoreApplication app;
   CommFPGA rs232;
   MovimCarro rover(&rs232);
   MovimPanTilt pantilt;
   CommmTcpIpServer comm;
   LogManager log;

   Establecer conexiones signal/slot. Ejemplo:
   QObject::connect(&comm,SIGNAL(NewMovementCarro(QVector2D)),
                    &rover,SLOT(OnMovementRequired(QVector2D)));

   QObject::connect(&rover,SIGNAL(PositionAvailable(QPointF)),
                    &comm,SLOT(OnPositionAvailable(QPointF)));

   QObject::connect(&rover,SIGNAL(AddToLog(QString,QString)),
                    &log,SLOT(OnLogFrom(Qstring,QString)) );
   ...
   Leer configuración inicial de cada elemento desde archivo XML. Ejemplo:
   QString xmlConfigTxt=ReadAllTextFromFile(argv[1]);
   rover.InitializeFromXml(GetXmlString(xmlConfigTxt,"Rover"));
   pantilt.IntializeFromXml(GetXmlString(xmlConfigTxt),"PanTilt"));
   comm.InitializeFromXml(GetXmlString(xmlConfigTxt),"Comm");
   ...
   return app.exec();
```

Gestiona puerto serie para comm con FPGA

maneja todo lo relacionado con el carro (motores de tracción, servo de dirección, trayectoria, etc.)

maneja todo lo relacionado con el pan-tilt

Gestiona comunicaciones WiFi con VISU, User XXX

Gestiona log para guardar eventos

Cuando llega una solicitud de nuevo movimiento de carro (User XXXX), activa la ejecución de dicho movimiento (rover)

Cuando se dispone de nuevos datos de posición de carro (rover), envía a los interesados para visualización

Cuando el carro tenga info para guardar en log (car), activa guardado

Programación orientada a objetos y eventos

- CORE: programa modo consola

MovimCarro.h

```
#includes necesarios

class MovimCarro : public QObject
{
   Q_OBJECT
private:
   Trajectory traj;
   MotorController controller;
   MotorNanotec motorLeft,motorRight;
   ServoMotor steering;
   QTimer timer;
public:
   MovimCarro();
   bool InitializeFromXml(const QString& xmlText);
   ...
public slots:
   void OnMovementRequired(QVector2D direction);
   void OnSetTimerInterval(int t_ms);
   void OnTimer();
   ...
signals:
   void PositionAvailable(QPointF position);
   ...
}
```

MovimCarro.cpp

```
#includes necesarios

bool MovimCarro::InitializeFromXml (...)
{
    controller.InitializeFromXml(xmlText,"Controller");
    motorLeft.Initialize…(...,&controller);
    ...
}
void MovimCarro::OnMovementRequired(QVector2D dir)
{
   traj.CalculateMotorSpeedsAndSteering(dir);
   motorLeft.SetSpeed(traj.GetRequiredSpeedLeft());
   motorRight.SetSpeed(traj.GetRequiredSpeedRight());
   steering.SetPosition(traj.GetRequiredSteeringDir());
}

void MovimCarro::OnTimer()
{
   float speedLeft,speedRight;
   QPointF newPos;

   speedLeft=motorLeft.GetCurrentSpeed();
   speedRight=motorRight.GetCurrentSpeed();
   newPos=traj.CalculateTraj(speedLeft,speedRight);

   emit PositionAvailable(newPos);
}
```

Programación orientada a objetos y eventos

- User-PC y/o User-Phone: programa modo GUI o cliente/servidor web

MainWindow.h

```
#includes necesarios

class MainWindow : public QMainWindow
{
   Q_OBJECT
private:
   CommTcpIpClient comm;

public:
   MainWindow();
   bool InitializeFromXml(const QString& xmlText);
...
public slots:
   ...
}
```

MainWindow.cpp

```
#includes necesarios

MainWindow::MainWindow(...)
{
   ui->setupUI(this);
   connect(&comm,SIGNAL(NewPosReceived(QPointF)),
           ui->qTrajWidget,SLOT(DrawNewPos(QPointF)));
   ui->qTrajWidget->Initialize(...);
}

void MainWindow::on_qTimerInterval_...clicked()
{
   TimerInterval dlg;
   dlg.SetValue(...);
   if (dlg.exec()==QDialog::Accepted)
      comm.SetTimerInterval(dlg.GetValue());
}
```

Programación orientada a objetos y eventos

- M5-STACK y UserPAD: Arduino SDK

En el mismo directorio que el programa principal

MainProgram.ino

```
#includes necesarios

WiFiClient _myClient;
...
MyClass _myClass;

void setup()
{
    Serial.begin(115200);
    WiFi.mode(WIFI_STA);
    WiFi.begin(ssid_name,ssid_pwd);
    ...
    _myClient.connect(server_ip,server_port);
}

void loop()
{
    if (Serial.available() )
    { ... }
    if (_myClient.available() )
    { ... }
}
```

MyClass.h

```
#includes necesarios

class MyClass
{
   ...
} ;
```

MyClass.cpp

```
#includes necesarios

Cpp convencional
Puede usar Serial, WiFi,
etc con los includes
adecuados
```

El main() de Arduino está escondido al programador, sólo puede usar setup() y loop(). Por ello, las variables que necesiten tiempo de vida permanente deben ser globales.

Para algunas clases (Serial, WiFi, etc.) el compilador de Arduino genera un objeto global con el mismo nombre de la clase

Ejemplo Comm User-PC → rPI





1

MainWindow::MainWindow()

client.Connect(xmlConfigText);

client.write("<Init><Client>USER-PC</Client></Init>");

ComuTcpServer:: ComuTcpServer() → server.listen(…);

ComuTcpServer::OnNewConnection()

connList.append(server.nextPendingConnection());

2

MainWindow::on_qSetTimer..._clicked()

ComuTcpClient::SetTimerInterval(int t_ms)

Componer xml

client.write(xml.toLatin1());

ComuTcpServer::onReadyRead()

Decode xml

Init

SetTimer

theConn->SetObjectName(name)

emit SetTimerInterval(t_ms);

MovimCarro::OnSetTimerInterval(t_ms);

Ejemplo Comm rPI → User-PC





1

MovimCarro::OnTimer()

emit PositionAvailable(newPos);

ComuTcpServer::OnPosAvailable(QPointF pos)

Componer xml

```
for (iConn=0;iConn<connList.count();iCon++)
{
   if (connList[iConn]->GetObjectName()==“USER-PC”)
      connList[iConn]->write(xml);
}
```

ComuTcpClient::onReadyRead()

Decode xml

emit NewPosReceived(pos);

TrajWidget::DrawNewPos (QPointF pos)

Ejemplo Comm User-PAD → rPI





setup()

client.connect(ip_server,port_server);

client.write("<Init><Client>USER-PAD</Client></Init>");

ComuTcpServer::OnNewConnection()

connList.append(server.nextPendingConnection());

loop()

Cada T ms

Leer sensores

Filtrar valores

Componer xml

client.write(xml);

ComuTcpServer::onReadyRead()

Decode xml

Init

MoveCar

theConn->SetObjectName(name)

emit NewMovementCarro(vector_dir)

MovimCarro::OnMovementRequired(QVector2D dir)



Coordinación e instalaciones (CORE+VISU)

Raspberry Pi

- Programa principal Qt modo consola (CORE+VISU)
  - Lectura de archivo de configuración y comunicación de configuración a subsistemas (CORE+VISU)
- Clases para gestión de:
  - Servidor comunicaciones TCP/IP con el resto de equipos (CORE+VISU)
  - Movimiento del carro, estado, posición, trayectoria (PATH)
  - Controladora y motores Nanotec para desplazamiento del carro (PATH)
  - Movimientos de pan-tilt de cámara, comunicación con FPGA (PWM-PanTilt)
  - Log en archivo (PWM-PanTilt)



Coordinación e instalaciones (CORE+VISU)

PC / tfno. móvil

- Programa principal Qt modo GUI (CORE+VISU)
- Clases para gestión de:
  - Opciones de configuración (diálogos) (CORE+VISU)
  - Cliente TCP/IP para comunicación con rPI, almacenamiento de resultados (CORE+VISU)
  - Visualización de estado y trayectorias (CORE+VISU , PATH)
  - Modificaciones para implantación en Android (CORE+VISU)



ESP32-S3 (USER-PAD) (*)

- Programa para adquisición de datos, filtrado, generación y comunicación de órdenes de movimiento, visualización de cámara

M5-Stack (visualización carro) (CORE+VISU)

- Programa para visualización del estado general, interfaz básico, y comunicación con rPI





FPGA

- Programa para comunicación serie y control de servos (PWM-PanTilt)

(*) Realizado por el profesor

Desarrollos preliminares

(CORE+VISU)

- Organización del software Raspberry Pi y coordinación. Parámetros necesarios en configuración
- Comunicación TCP/IP clientes-servidor, modo texto, con codif/decod incluyendo info para destinatario de los mensajes
- Aspecto general y pantallas deseadas para la aplicación PC y móvil, búsqueda de librerías para visualización
- Presentación y menús en M5-STACK

(PATH)

- Ecuaciones que relacionan trayectorias y movimientos de motores carro + servo dirección
- Configuración de motores NanoTec con programa disponible y comunicación RS-485 con los motores

(PWM)

- Comunicación serie con FPGA movim, generación de pulsos para movim

(USER-PAD) (realizado por el profesor)

- Adquisición de señales Joystick y botones, comunicación con CORE
- Adquisición imagen 3D en rPI, envío imagen, visualización en pantalla USER-PAD

Documentación a generar (04/11/2026)

- Listado de clases y personas que las desarrollarán
- Protocolos XML: datos necesarios en archivos de configuración

USER-PC

```
<Config>
  <CORE-Server>
    <Ip>192.168.100.25</Ip>
    <Port>54545</Port>
  </CORE-Server>
  …
</Config>
```

CORE

```
<Config>
  <CORE-Server>
    <Port>54545</Port>
  </CORE-Server>
  <Motors>
    <Comm>/dev/ttyS0</Comm>
    <Tm_ms>100</Tm_ms>
    <Left> <Id> 1 </Id> … </Left>
    <Right> <Id> 2 </Id> … </Right>
  </Motors>
  <Path>
    <WheelDiam_mm>50</Whee…>
    …
  </Path>
  …
</Config>
```

- Protocolos XML: de intercambio de información en comu TCP/IP

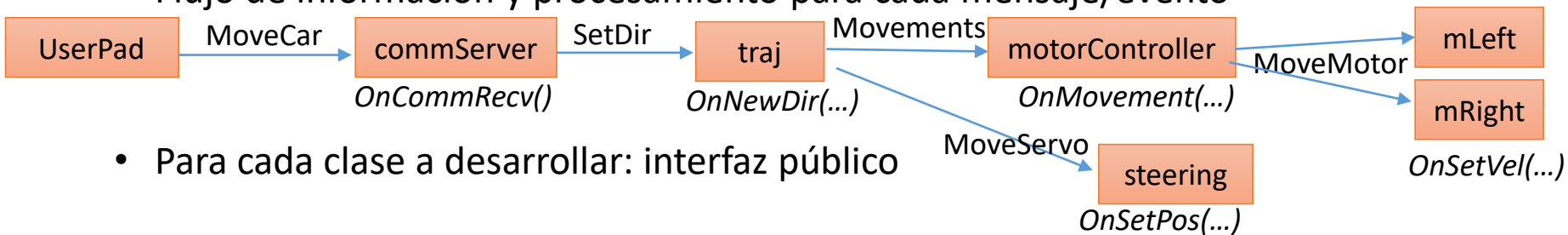
ALL → CORE `<Init><Client>Nombre Cliente</Init></Client>` → 1er mensaje de todos los clientes

USER-PAD → CORE `<MoveCar><VelX>…</VelX><VelY>…</VelY></MoveCar>`

USER-PAD → CORE `<Elev><PosVert>…</PosVert></Elev> <Pulse>Off</Pulse>`

→ Especificar campos y unidades de medida

- Eventos de interés para cada programa:
  - Clase generadora y receptora
  - Argumentos necesarios para cada pareja señal/slot
- Flujo de información y procesamiento para cada mensaje/evento



- Para cada clase a desarrollar: interfaz público